

1장

FastAPI 소개

이 장에서는 파이썬 기반의 웹 프레임워크인 FastAPI를 소개합니다. FastAPI가 등장한 배경과 핵심 특징을 정리하고 Flask(플라스크), Django(장고) 같은 다른 프레임워크와 비교해 어떤 점에서 다른지 확인합니다. 또한 FastAPI의 기술적 특성이 개발 생산성과 코드 품질에 어떤 영향을 주는지 살펴보고 이 책의 실습을 위해 FastAPI 개발 환경을 설정합니다.

1.1

FastAPI 개요

FastAPI를 이해하려면 먼저 웹 애플리케이션이 어떤 구조로 동작하는지 살펴봐야 합니다. 우리가 사용하는 대부분의 웹 서비스는 클라이언트-서버 모델(client-server model)을 기반으로 동작하며, 웹 애플리케이션은 이 구조를 바탕으로 만들어집니다.

1.1.1 클라이언트-서버 모델

인터넷에서 데이터를 주고받는 대부분의 서비스는 클라이언트와 서버로 역할을 나누어 통신합니다. 클라이언트(client)는 정보를 요청하는 쪽으로, 사용자가 웹 브라우저나 모바일 앱을 실행해 어떤 기능을 사용했다면 브라우저와 앱은 클라이언트가 됩니다. 서버(server)는 요청을 받아 처리한 후 결과를 응답하는 쪽으로, 검색 결과를 제공하거나 영상 데이터를 전달하는 작업은 모두 서버에서 처리됩니다.

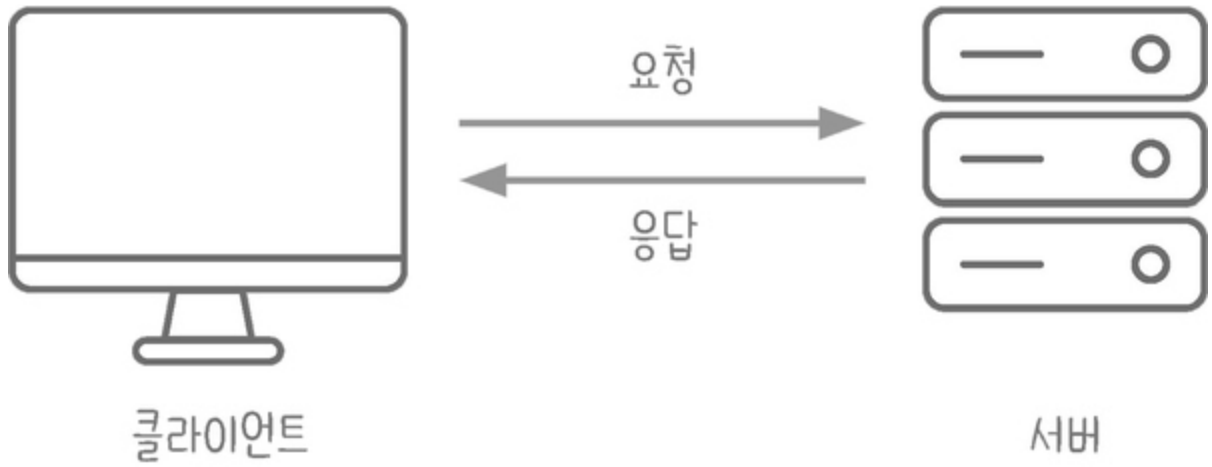


그림 1-1 클라이언트-서버 모델

클라이언트와 서버는 물리적인 컴퓨터를 의미하기도 하고, 그 안에서 실행되는 프로그램을 의미하기도 합니다. 예를 들어 사용자의 PC는 클라이언트가 될 수 있고, 그 안에서 실행되는 크롬 브라우저 역시 클라이언트가 될 수 있습니다. 중요한 것은 누가 요청을 보내고 누가 처리하는냐이며 이에 따라 요청을 보내면 클라이언트, 요청을 받아 처리하면 서버입니다.

1.1.2 FastAPI 소개

FastAPI는 웹 애플리케이션의 서버 측 로직을 구현하기 위한 파이썬 웹 프레임워크입니다. 웹 애플리케이션은 프런트엔드와 백엔드로 나누어 개발합니다. 프런트엔드(frontend)는 클라이언트 쪽에서 화면을 구성해 사용자와 상호작용하는 부분이고, 백엔드(backend)는 서버 쪽에서 요청을 받아 처리하고 데이터를 관리하는 부분입니다. FastAPI는 백엔드 영역을 개발할 때 사용하는 도구로, 클라이언트로부터 요청을 받아 처리하고 데이터베이스와 통신한 뒤 그 결과를 응답하는 역할을 수행합니다.



그림 1-3 FastAPI 로고

FastAPI는 파이썬 3.10 이상에서 동작하며 웹 애플리케이션을 일정한 구조 안에서 체계적으로 개발할 수 있도록 설계되었습니다. 요청·응답 처리, 라우팅, 데이터 검증, 인증과 같은 반복적이고 표준화된 서버 측 기능을 간결한 코드로 구현할 수 있게 해 줍니다. 덕분에 개발자는 서버의 구조를 직접 구성하는 데 시간을 쓰기보다 웹 애플리케이션의 핵심 기능을 구현하는 데 집중할 수 있으며, 그로 인해 전체 개발의 생산성을 향상시킬 수 있습니다.

1.1.3 FastAPI의 발전 과정

FastAPI는 2018년 콜롬비아 출신 개발자 세바스티안 라미레즈(Sebastián Ramírez)가 오픈소스로 공개했습니다. 이후 오픈소스 커뮤니티의 지지를 받으며 성장했고, 2023년에는 라미레즈가 세쿼이아 캐피털(Sequoia Capital)의 오픈소스 펠로우십(Open Source Fellowship)의 수혜자로 선정되어 FastAPI 개발에 집중할 수 있도록 자금을 지원받았습니다. 또한 FastAPI의 핵심 구성 요소인 Pydantic(파이단틱) 역시 같은 해 투자를 유치하며 두 프로젝트(FastAPI와 Pydantic)의 연계가 더욱 강화되었습니다. 이러한 지원은 FastAPI가 단순히 개인 프로젝트를 넘어 커뮤니티와 산업계의 관심을 받으며 꾸준히 발전하고 있음을 보여 줍니다.

1.2

FastAPI의 특징

FastAPI에서 Fast는 두 가지 의미를 가집니다. 첫 번째는 개발 속도가 빠르다는 의미의 Fast to code이고, 두 번째는 실행 속도가 빠르다는 의미의 Fast to run입니다. 이러한 속도를 가능하게 하는 FastAPI의 대표적인 특징을 살펴보겠습니다.

1.2.1 타입 힌트를 이용한 데이터 검증

타입 힌트(type hint)는 변수, 함수의 매개변수와 반환값 등에 데이터 타입을 명시하는 문법입니다. `:`(콜론) 뒤에 데이터 타입을 작성하는 방식으로 표현합니다.

- `age: int` → `age`는 정수형(`int`)
- `name: str` → `name`은 문자열(`str`)
- `is_active: bool` → `is_active`는 논리형(`bool`)

파이썬은 기본적으로 동적 타입 언어이므로 변수의 타입을 미리 선언하지 않아도 됩니다. 그러나 타입 힌트를 사용하면 코드의 의도를 더 명확하게 드러낼 수 있습니다. 다음은 함수의 매개변수와 반환값에 타입 힌트를 적용한 예입니다.

```
def greeting(name: str) → str:
    return f"Hello, {name}!"
```

- `name: str`은 매개변수 `name`이 문자열임을 의미합니다.
- `→ str`은 함수의 반환값이 문자열임을 의미합니다.

1.2.2 API 문서 자동 생성

클라이언트와 서버가 서로 통신하려면 어떤 경로로 요청을 보내고 어떤 형식으로 응답을 받는지에 대한 약속이 필요합니다. 이 약속을 정리한 문서를 API 문서라고 합니다. API 문서에는 요청 경로, 필요한 매개변수, 응답 데이터 구조, 인증 방식, 오류 코드 등이 포함됩니다.

일반적으로 API 문서는 개발자가 별도로 작성하고 관리합니다. 그러나 코드와 문서를 따로 관리하면 기능이 수정될 때 문서가 최신 상태를 반영하지 못하는 문제가 발생할 수 있습니다. 문서와 실제 동작이 어긋나면 협업 과정에서 혼란이 생깁니다.

FastAPI는 이러한 문제를 해결하기 위해 코드를 기반으로 API 문서를 자동으로 생성합니다. 개발자가 정의한 경로(함수), 입력 데이터의 타입, 반환 데이터 구조 등을 분석해 OpenAPI 표준 형식의 명세를 자동으로 만듭니다. 즉, 별도의 문서 작성 작업 없이 코드만으로 API 명세가 완성됩니다.

1.2.3 비동기 방식 지원

서버는 동시에 여러 클라이언트의 요청을 처리합니다. 그런데 만약 요청을 하나씩 순차적으로 처리하면 앞선 작업이 끝날 때까지 다음 작업이 대기하게 되어 전체 응답 속도가 느려질 수 있습니다. 특히 데이터베이스 조회나 외부 API 호출처럼 시간이 걸리는 작업이 많을수록 이러한 지연은 더 크게 느껴집니다.

이 문제를 해결하기 위해 비동기 방식을 이용합니다. 비동기 방식은 한 작업이 완료될 때까지 무조건 기다리는 대신 그동안 다른 작업을 먼저 처리합니다. 그 결과 서버는 여러 요청을 효율적으로 병행 처리할 수 있고 그로 인해 자원의 활용도와 응답 속도를 향상할 수 있습니다.

1.3

파이썬 웹 프레임워크 비교

파이썬 웹 프레임워크에는 FastAPI 외에도 Flask(플라스크)와 Django(장고)가 있습니다. 이들과 FastAPI의 차이점을 살펴봅시다.

1.3.1 Flask

Flask는 2010년에 오스트리아의 개발자 아르민 로나허(Armin Ronacher)가 만든 경량화 마이크로 웹 프레임워크입니다. 매우 간단한 구조로 최소한의 핵심 기능만 제공하며, 필요에 따라 다양한 확장 기능을 플러그인 형태로 추가해 프로젝트를 자유롭게 구성할 수 있습니다.



그림 1-6 Flask 로고

다만, Flask는 기본적으로 비동기 처리를 지원하지 않고, 타입 힌트를 활용한 데이터 자동 검증과 API 문서화 기능이 내장되어 있지 않아 별도 라이브러리를 추가해야 이러한 기능을 사용할 수 있습니다.

Flask는 주로 소규모 웹 애플리케이션, REST API 서버, 빠른 시제품 개발에서 사용합니다. 간단하고 유연한 구조 덕분에 웹 개발을 처음 시작하는 초보자도 부담 없이 사용할 수 있으며, 최소 기능만으로 프로젝트를 시작해 필요한 기능을 직접 추가하려는 개발자에게도 좋은 선택지입니다.

1.3.2 Django

Django는 2005년 미국의 로렌스 저널 월드(Lawrence Journal-World) 신문사의 개발자 두 명이 주도하여 만든 파이썬 기반의 풀스택 웹 프레임워크입니다. 배터리 포함(batteries included, 개발에 필요한 대부분의 기능과 도구가 프레임워크에 이미 내장되어 있는 것)이라는 철학을 바탕으로 다양한 기능을 기본으로 제공하며, 강력한 ORM(Object-Relational Mapping), 인증 시스템, 관리자 인터페이스, 템플릿 엔진 등 웹 개발에 필요한 거의 모든 요소를 내장하고 있어 대규모 웹 애플리케이션이나 복잡한 프로젝트에 적합합니다.



그림 1-7 Django 로고

1.3.3 Flask, Django, FastAPI 비교

특정 기술에 대한 개발자의 관심을 가늠할 수 있는 지표인 깃허브 스타(star) 수를 보면, FastAPI가 최근 몇 년 사이 빠르게 인기를 얻고 있습니다. 2018년 출시 이후 스타 수가 급격히 늘어나 Django나 Flask보다 빠른 성장세를 보이고 있으며, 2026년 4월 기준으로 스타 수가 9.6만 개를 넘어 전체 파이썬 웹 프레임워크 중 가장 높은 수를 기록했습니다. 이는 FastAPI가 최신 파이썬 기능을 적극 활용하면서도 빠른 개발과 높은 성능을 제공해 웹 프레임워크 시장에서 영향력을 빠르게 확대하고 있음을 보여줍니다.

1.4

실습 환경 설정

FastAPI를 실습하기 위한 개발 환경을 구축하겠습니다. 이는 다음 순서로 진행합니다.

1. 파이썬을 설치합니다. FastAPI는 파이썬으로 만들어진 프레임워크이기 때문에 파이썬이 설치되어 있어야 FastAPI의 핵심 라이브러리와 필요한 의존성을 실행할 수 있습니다.

2. 파이썬 개발용 IDE인 파이참을 설치합니다. 파이참은 가상환경 관리, 코드 작성과 실행, 디버깅을 보다 편리하게 수행할 수 있도록 해 줍니다.

3. FastAPI 프로젝트를 생성합니다.

4. 파이참에 가상환경을 생성하여 프로젝트별로 독립된 개발 환경을 구성합니다.

5. 첫 애플리케이션을 만들고 실행합니다. 애플리케이션을 실행하면 서버가 작동하며, 자동으로 생성된 API 문서(Swagger UI, ReDoc)를 통해 API로 요청을 보내고 응답받을 수 있습니다.

1.4.1 파이썬 설치하기

FastAPI는 파이썬 3.10 이상에서 실행할 수 있으며, 이 책에서는 최신 안정 버전인 파이썬 3.13을 사용합니다. 설치 방법은 윈도우와 맥OS로 나눠 설명합니다.

1.4.2 파이참 설치하기

파이참(PyCharm)은 젯브레인스(JetBrains)에서 개발한 파이썬 전용 IDE로, 코드 작성, 실행, 디버깅, 테스트, 패키지 관리, 버전 관리 등을 한곳에서 수행할 수 있습니다. 파이참은 개발 생산성을 높이는 다음 기능을 제공합니다.

- 코드 자동 완성과 문법 검사로 작성 중인 코드의 오류를 실시간으로 확인할 수 있습니다.
- 프로젝트별 독립된 가상환경을 설정하여 손쉽게 패키지를 관리할 수 있습니다.
- 코드 실행을 멈추고 변수 값을 직접 확인할 수 있는 디버깅 기능을 제공합니다.
- 깃, 깃허브 같은 버전 관리 도구와 연동할 수 있어 개발 생산성을 높여 줍니다.

1.4.3 프로젝트 생성하기

1 바탕화면(또는 원하는 경로)에 hello_fastapi라는 이름으로 새 폴더를 만듭니다. 이 폴더가 앞으로 FastAPI 실습 프로젝트의 작업 공간입니다.

1.4.4 가상환경 만들기

가상환경(virtual environment)이란 프로젝트별로 독립된 패키지 설치 공간을 제공하는 격리된 환경을 말합니다. 가상환경을 사용하면 프로젝트마다 필요한 패키지 버전을 별도로 관리할 수 있어 다른 프로젝트와의 충돌을 방지하고 안정적인 개발 환경을 유지할 수 있습니다. 가상환경 생성과 활성화는 파이썬 내장 모듈 `venv`를 사용합니다.

> 💡 **[최신 FastAPI CLI 권장 가이드] fastapi dev 사용**
> FastAPI 0.111.0 버전부터 자체 개발 서버 기동용 CLI 도구인 `fastapi dev`가 도입되었습니다.
> 기존의 구버전 방식인 `uvicorn main:app --reload` 대신, 최신 표준이자 개발 디버깅에 최적화된 `fastapi dev main.py` 명령어로 서버를 기동하는 것이 적극 권장됩니다.
> (※ 배포용 프로덕션 서버 실행 시에는 `fastapi run main.py`를 사용합니다.)

1.4.5 애플리케이션 실행하기

가상환경을 설정하고 FastAPI 설치를 완료했으니 이제 간단한 FastAPI 애플리케이션을 만들어 실행해 보겠습니다.

마무리

1 클라이언트-서버 모델

요청을 보내는 클라이언트와 요청을 처리해 응답하는 서버가 역할을 분리하여 통신하는 구조입니다.

2 UI와 API

① UI는 버튼, 입력창, 아이콘처럼 사용자가 직접 보고 조작하는 화면 요소로, 클라이언트는 사용자와 가까운 위치에서 실행되며 UI를 통해 상호작용합니다.

② API는 두 프로그램이 상호작용하기 위해 정해 둔 규칙으로, 클라이언트 프로그램과 서버 프로그램 사이 통신은 API를 통해 이루어집니다.

3 FastAPI 소개

파이썬 3.10 이상에서 동작하는 웹 프레임워크로, 빠른 개발 속도와 높은 실행 성능을 갖췄습니다.